

CSE 3421: Software Engineering

Sherajul Arifin
Lecturer, Department of Computer Science and Engineering,
United International **University**

Current Trends in Software Engineering

What is Software Engineering Today?

Software engineering in current days is more important than ever. It is the force behind all the apps, websites, and digital systems we use every day.

- From code-centric to **data + AI + automation**
- Software as **services** (SaaS, microservices, APIs)
- Developer roles are evolving with AI tools

Modern Software Development

- Web Applications
- Mobile Applications
- Cloud-Native & Serverless Systems
- AI-Driven Applications
- Edge & IoT Systems
- XR (AR/VR) Applications

Web and Mobile Development Trends

Modern Web Development



Modern web development is the process of creating interactive websites and applications using a wide array of front-end and back-end technologies, emphasizing responsive design, security, and efficiency.

- Frontend frameworks: React, Vue, Svelte
- Backend: Node.js, Deno, Go, Python FastAPI
- Full-stack: Next.js, Remix, Nuxt

Modern Mobile Development

Modern mobile development focuses on building apps with a mix of native and cross-platform frameworks while incorporating trends like AI integration and declarative UI to improve efficiency and user experience.

- Native: Swift (iOS), Kotlin (Android)
- Cross-platform: Flutter, React Native, Kotlin Multiplatform
- PWAs (installable websites acting as mobile apps)

AI in Mobile Apps

AI in mobile apps involves integrating AI, ML, and NLP(natural language processing) into mobile applications to perform tasks like pattern recognition, prediction, and automation. It enhances functionality with voice interactions, visual AI, and personalized experiences, making apps smarter and more user-friendly.

- Voice-based interaction (SiriKit, ChatGPT API)
- Visual AI (MLKit, TensorFlow Lite)
- Personalized user experiences

APIs and Integration

Application Programming Interface (API)



- API is an intermediary that allows two applications to talk to each other.
- API defines endpoints, and valid request and response formats.
- Different request methods: get, post, put, delete
- Response formats include JSON, XML
- Sample API example: https://jsonplaceholder.typicode.com/

Types of API

- **Open APIs:** also known as external or public APIs, are available to developers and other users with minimal restrictions.
- **Internal APIs**: They are designed to be hidden from external users. They are used within different teams of a company to share resources.
- **Partner APIs**: Partner APIs are technically similar to open APIs, but they feature restricted access, often controlled through a third-party API gateway.
- **Composite APIs**: Composite APIs allow developers to access several endpoints in one call. These could be different endpoints of a single API, or they could be multiple services or data sources.

API Protocols

- **REST (representational state transfer):** a web service can only be treated as a RESTful service if it follows the constraints of being
    1. Client Server
    2. Stateless
    3. Cacheable
    4. Layered System
    5. Uniform Interface

  REST requires low bandwidth, and response format can be plain text, HTML, XML, JSON, etc.

- **SOAP (simple object access protocol):** It requires high bandwidth and can only work with XML format

Cloud-Native & DevOps

Cloud-Native Development

A modern approach to building and running applications that fully leverage **cloud computing technologies**, enabling applications to be scalable, resilient, portable, and easy to manage.

- Containers → Enable portability across environments
- Kubernetes → Provides orchestration for complex applications
- Serverless → Offers simplicity by abstracting infrastructure

CI/CD in Practice



CI/CD in practice involves automating the software development lifecycle, using a CI/CD pipeline where developers frequently merge code to a central repository (Continuous Integration), which automatically triggers builds and tests. If tests pass, the code can be automatically released to various environments, with Continuous Delivery.

- **Continuous integration** = automated testing on every commit
- **Continuous delivery** = automated deployment
- Tools: Jenkins, GitHub Actions, GitLab CI

AI-Augmented Software Engineering

AI in Software Development

The integration of AI into the software development process to automate tasks, enhance code quality, and accelerate development.

AI helps developers write smarter code, detect bugs early, and optimize workflows, making software development faster, more reliable, and more efficient.

- Code generation (GitHub Copilot, ChatGPT)
- AI for debugging, documentation, and optimization

LLMs in Software Engineering

Large Language Models (LLMs) are being integrated into software engineering for tasks like code generation, bug fixing, and architectural design suggestions, while also impacting the role of developers by automating certain tasks and demanding new skills

- Understand and generate human-like text, including code
- ChatGPT, Gemini, Claude

Integration through APIs:

- Easily embed AI capabilities into software tools and workflows
- OpenAI, Hugging Face

Code Understanding & Generation

- Assist developers by writing code, suggesting improvements, detecting bugs, and even automating repetitive tasks

Prompt Engineering for Developers



The practice of crafting effective inputs (prompts) for AI models, especially Large Language Models (LLMs), to get accurate, relevant, and useful outputs.

- Involves choosing words, structure, and context carefully.
- Helps AI understand the task better and produce high-quality results.
- Chain-of-thought prompting can be used
- Used in tasks like code generation, content creation, and data analysis.
- Limitations: hallucination, privacy

Emerging & Responsible Trends

Edge and IoT Computing



Edge and IoT computing work together by using edge computing to process data locally near the Internet of Things (IoT) devices that generate it, rather than sending it to a central cloud. This approach reduces latency, enhances security, and decreases bandwidth use.

- Low latency, offline capabilities
- Real-time analytics on devices
- Use cases: autonomous vehicles, smart homes

Green Software Engineering



Green software engineering is a discipline focused on creating software with minimal environmental impact, prioritizing energy efficiency and reduced carbon emissions throughout the software lifecycle.

- Optimize CPU & energy usage
- Efficient cloud scaling
- Sustainability metrics

Responsible & Ethical AI in SE



The practice of developing and using AI systems in a way that is fair, transparent, accountable, and aligned with ethical standards throughout the software development lifecycle.

- **Fairness**: Avoid biases in AI models and decisions
- **Transparency**: Make AI decision-making understandable and explainable
- **Accountability**: Developers are responsible for AI outputs and impacts
- **Privacy & Security:** Protect sensitive user data and maintain confidentiality
- **Sustainability**: Design AI systems that are energy-efficient and socially responsible

Any Questions?

Thank You!